REȚELE DE CALCULATOARE

Săptămâna 7

Comunicația prin socket-uri: definiții, clasificare și programare client-server în Java peste TCP (Transmission Control Protocol) și UDP (User Datagram Protocol)

Curs • Seminar • Laborator (variantele Windows și Linux)

Cuprins

Obiective de învățare și legătura cu săptămâna 6

Partea I — Curs

1. De la serviciile de transport la aplicații
2. Socket-uri: definiții
3. Clasificarea socket-urilor
4. API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket și secvențele de apeluri
5. Programarea în rețea în Java
6. Comunicația client-server TCP (Transmission Control Protocol) în Java
7. Comunicația client-server UDP (User Datagram Protocol) în Java
8. Programarea TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol); bune practici
9. Rezumat și termeni-cheie

Partea a II-a — Seminar: analiză, depanare și proiectarea protocoalelor

Partea a III-a — Laborator

Laboratorul 7-W: socket-uri Java în Windows

Laboratorul 7-L: socket-uri Java în Linux

Teme de programare (ambele variante)

Anexă — soluție de referință și bibliografie

Obiective de învățare și legătura cu săptămâna 6

În săptămâna 6 am studiat nivelul transport din punctul de vedere al rețelei: porturi, antetele UDP (User Datagram Protocol) și TCP (Transmission Control Protocol), stabilirea conexiunii în trei pași, ferestre glisante și controlul congestiei. **Săptămâna aceasta (săptămâna 7)** privește același nivel din **punctul de vedere al programatorului**. Aplicațiile nu construiesc niciodată singure segmente TCP (Transmission Control Protocol); ele cer sistemului de operare un **socket** și citesc și scriu date prin el. Definim socket-urile, le clasificăm, studiem API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket și scriem programe client-server TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) complete în **Java**, pe care apoi le compilăm și le testăm în **Windows** și **Linux**.

La finalul acestei săptămâni, studenții ar trebui să poată:

1. Să definească un socket, o adresă de socket și o conexiune și să le lege de 5-tuplul din săptămâna 6.
2. Să clasifice socket-urile după familia de adrese, tip, rol, modelul I/O (Input/Output – intrare/ieșire) și modul de livrare.
3. Să descrie ordinea apelurilor API (Application Programming Interface – interfață de programare a aplicațiilor) socket pentru clienți și servere TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) și să lege fiecare apel de pachetele pe care le generează.
4. Să folosească clasele Java `InetAddress`, `Socket`, `ServerSocket`, `DatagramSocket` și `DatagramPacket`.
5. Să scrie, să compileze și să ruleze servere TCP (Transmission Control Protocol) iterative și concurente și servere UDP (User Datagram Protocol), împreună cu clienții lor.
6. Să proiecteze un protocol de aplicație simplu, bazat pe text, și să trateze delimitarea mesajelor peste un flux de octeți TCP (Transmission Control Protocol).
7. Să diagnosticheze erorile tipice ale socket-urilor (conexiune refuzată, adresă în uz, expirări, blocări de firewall) în Windows și Linux.

Partea I — Curs

1. De la serviciile de transport la aplicații

Aplicațiile de rețea sunt construite din procese care schimbă mesaje. Cea mai răspândită organizare este **modelul client-server**: un proces **server** rulează permanent la o adresă și un port cunoscute, așteaptă cereri și le răspunde; un proces **client** inițiază comunicația atunci când are nevoie de un serviciu. În modelul **peer-to-peer**, fiecare proces este simultan client și server. În ambele modele, procesele accesează nivelul transport prin aceeași interfață: **API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket**, oferit de sistemul de operare.

Tabelul 1. *Rolurile în modelul client-server*

Caracteristică	Server	Client
Adresă și port	Fixe, binecunoscute (de ex. portul 80, 443, 5000)	De obicei oricare; OS (Operating System – sistem de operare) alocă un port efemer
Durata de viață	Rulează continuu, așteptând cereri	Rulează când utilizatorul are nevoie de serviciu
Inițiativă	Pasiv: așteaptă (listen / receive)	Activ: se conectează / trimite primul
Concurență	Servește mulți clienți (iterativ sau concurent)	De obicei comunică cu un singur server o dată

2. Socket-uri: definiții

> **Definiție**
>
> Un **socket** este un capăt al unui canal de comunicație bidirecțional între două procese, creat și gestionat de sistemul de operare. Aplicația îl identifică printr-un **handle** (un *descriptor de fișier* în Linux/Unix, un handle *SOCKET* în Windows, un obiect `Socket` în Java); rețeaua îl identifică prin **adresa de socket** (adresă IP (Internet Protocol), port) și prin protocolul de transport.

- **Adresa de socket** — perechea (adresă IP (Internet Protocol), port), de ex. `192.168.1.10:5000`. În Java: `InetSocketAddress`.
- **Conexiunea** — în TCP (Transmission Control Protocol), asocierea a două adrese de socket, identificată prin 5-tuplu (protocol, IP (Internet Protocol) local, port local, IP (Internet Protocol) la distanță, port la distanță).
- **Socket de ascultare (pasiv)** — un socket TCP (Transmission Control Protocol) de server care doar acceptă cereri de conexiune; nu transportă niciodată date.
- **Socket conectat** — un socket TCP (Transmission Control Protocol) legat de o anumită conexiune; el transportă datele. Un server are un socket de ascultare plus câte un socket conectat pentru fiecare client.
- **API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket** — mulțimea apelurilor de sistem folosite pentru a crea și utiliza socket-uri. A apărut în **4.2BSD (4.2 Berkeley Software Distribution) Unix (1983)** sub numele de *Berkeley sockets*, a fost standardizat în **POSIX (Portable Operating System Interface)** și a fost preluat în Windows ca **Winsock** (Windows Sockets, 1992). Pachetul Java `java.net` (din JDK (Java Development Kit – kit de dezvoltare Java) 1.0, 1996) încapsulează același API (Application Programming Interface – interfață de programare a aplicațiilor) în clase.

O imagine utilă: dacă gazda este o clădire, iar adresa IP (Internet Protocol) este adresa ei poștală, porturile sunt cutiile poștale numerotate din hol, iar un socket este ușa anume prin care un proces își trimite și își primește corespondența. Tot ce se află deasupra socket-ului ține de aplicație; tot ce se află sub el (TCP

(Transmission Control Protocol)/UDP (User Datagram Protocol), IP (Internet Protocol), driverul ține de sistemul de operare.

3. Clasificarea socket-urilor

Tabelul 2. *Clasificarea socket-urilor*

Criteriu	Clase	Observații / exemple
Familia de adrese (domeniul de comunicație)	AF_INET (IPv4 (Internet Protocol versiunea 4)) AF_INET6 (IPv6 (Internet Protocol versiunea 6)) AF_UNIX / AF_LOCAL AF_PACKET, AF_BLUETOOTH, …	IPv4 (Internet Protocol versiunea 4) și IPv6 (Internet Protocol versiunea 6) se folosesc pentru comunicația în rețea; AF_UNIX conectează procese de pe aceeași gazdă printr-o cale din sistemul de fișiere (suportat în Linux și în Windows 10 1803+); AF_PACKET (Linux) oferă acces la cadrele nivelului legătură de date
Tipul socket-ului (serviciul)	SOCK_STREAM SOCK_DGRAM SOCK_RAW SOCK_SEQPACKET	Stream → TCP (Transmission Control Protocol): flux de octeți fiabil și ordonat. Datagram → UDP (User Datagram Protocol): mesaje independente. Raw → acces direct la IP (Internet Protocol) (ping, traceroute, instrumente de pachete; necesită drepturi de administrator). Seqpacket → mesaje fiabile (SCTP (Stream Control Transmission Protocol))
Conexiune	Orientat pe conexiune Fără conexiune	Socket-urile TCP (Transmission Control Protocol) trebuie să se conecteze înainte de transferul datelor; socket-urile UDP (User Datagram Protocol) pot trimite oricând către orice adresă
Rol	Server (pasiv) Client (activ)	Server: socket → bind → listen → accept. Client: socket → connect
Modelul I/O (Input/Output – intrare/ieșire)	Blocant Neblocant Multiplexat Asincron	Apelurile blocante așteaptă (implicit în java.net). Apelurile neblocante revin imediat. Multiplexare: select/poll/epoll (Linux), IOCP (Input/Output Completion Ports – porturi de finalizare I/O) (Windows), Java NIO (New Input/Output – noua bibliotecă de intrare/ieșire) `Selector`. Asincron: callback-uri la finalizare (Java `AsynchronousSocketChannel`)
Modul de livrare	Unicast Broadcast Multicast	Broadcast și multicast sunt posibile doar cu socket-uri datagramă (Java `DatagramSocket` cu broadcast activat, `MulticastSocket`)

În acest curs folosim **socket-uri Internet** (AF_INET / AF_INET6) de cele două tipuri principale: **socket-uri stream (TCP (Transmission Control Protocol))** și **socket-uri datagramă (UDP (User Datagram Protocol))**, în mod **blocant**, cel mai simplu model de înțeles.

4. API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket și secvențele de apeluri

Tabelul 3. *Primitivele Berkeley sockets*

Primitivă (C / POSIX (Portable Operating System Interface))	Folosită de	Scop	Efect în rețea
socket()	TCP (Transmission Control Protocol), UDP (User Datagram Protocol)	Creează un socket de o anumită familie și un anumit tip; întoarce un handle	Niciunul
bind()	Servere (și opțional clienți)	Asociază o adresă IP (Internet Protocol) locală și un port	Niciunul

Primitivă (C / POSIX (Portable Operating System Interface))	Folosită de	Scop	Efect în rețea
listen()	Server TCP (Transmission Control Protocol)	Marchează socket-ul ca pasiv; creează coada conexiunilor în așteptare (backlog)	Portul apare în starea LISTEN
accept()	Server TCP (Transmission Control Protocol)	Preia din coadă o conexiune stabilită; întoarce un nou socket conectat	Stabilirea conexiunii a fost finalizată de nucleu
connect()	Client TCP (Transmission Control Protocol) (opțional UDP (User Datagram Protocol))	Deschide o conexiune către o adresă de socket la distanță	TCP (Transmission Control Protocol): trimite SYN (Synchronize – sincronizare), așteaptă SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare), trimite ACK (Acknowledgment – confirmare)
send() / recv() write() / read()	TCP (Transmission Control Protocol) (UDP (User Datagram Protocol) conectat)	Trimite / primește octeți pe un socket conectat	Segmente de date și ACKs (Acknowledgments – confirmări)
sendto() / recvfrom()	UDP (User Datagram Protocol)	Trimite / primește o datagramă către / de la o adresă explicită	O datagramă UDP (User Datagram Protocol) pentru fiecare apel
shutdown()	TCP (Transmission Control Protocol)	Închide un singur sens (semi-închidere)	Trimite FIN (Finish – terminare)
close() / closesocket()	TCP (Transmission Control Protocol), UDP (User Datagram Protocol)	Eliberează socket-ul	TCP (Transmission Control Protocol): FIN (Finish – terminare) (sau RST (Reset – resetare) dacă au rămas date necitite)

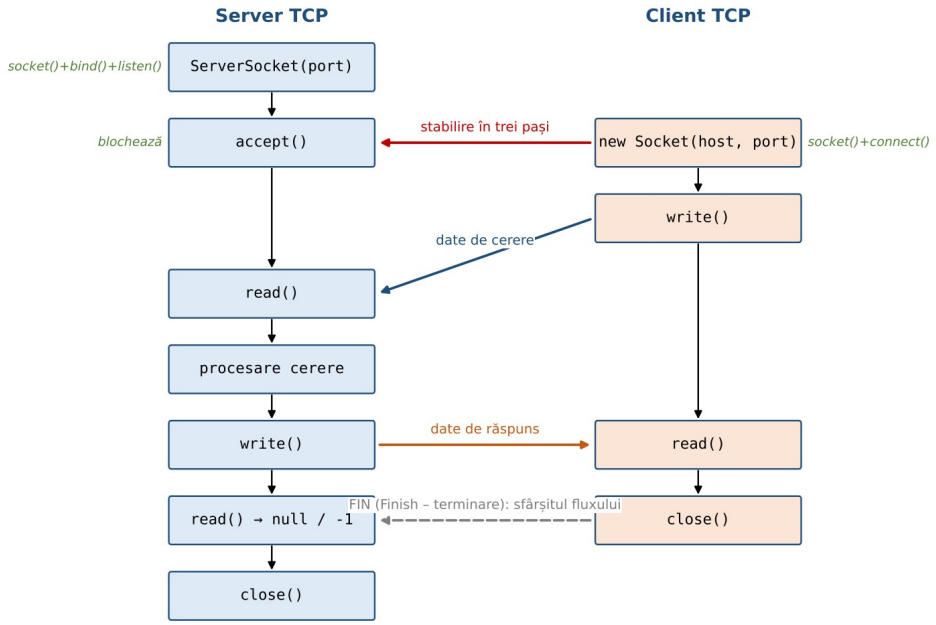


Figura 1. *Secvența apelurilor socket pentru un client și un server TCP (Transmission Control Protocol) (numele metodelor Java, cu apelurile POSIX (Portable Operating System Interface) corespunzătoare în cursiv)*

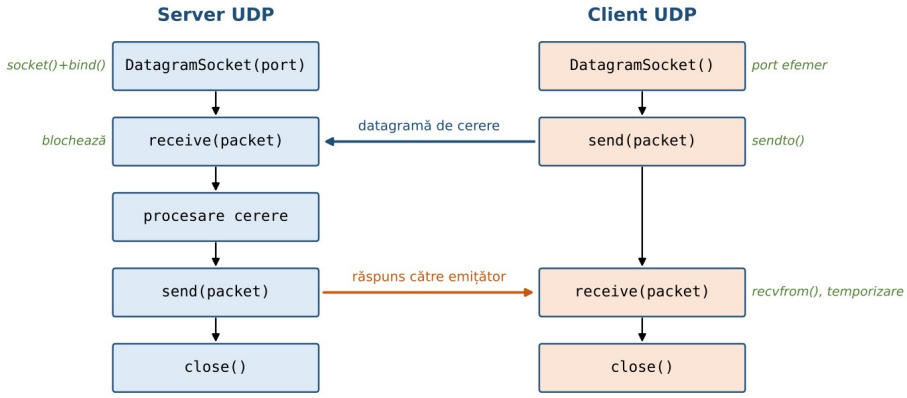


Figura 2. *Secvența apelurilor socket pentru un client și un server UDP (User Datagram Protocol) — nu există stabilirea conexiunii*

5. Programarea în rețea în Java

Java oferă programare în rețea pe mai multe niveluri. Pachetul **java.net** conține clasele clasice de socket-uri blocante folosite în acest curs; **java.nio.channels** adaugă I/O (Input/Output – intrare/ieșire) neblocant și multiplexat; clase de nivel mai înalt, precum `java.net.http.HttpClient`, implementează protocoale de aplicație complete.

Tabelul 4. *Principalele clase din java.net pentru programarea cu socket-uri*

Clasă	Scop	Metode principale
InetAddress	O adresă IP (Internet Protocol) (IPv4 (Internet Protocol versiunea 4) sau IPv6 (Internet Protocol versiunea 6)), cu rezolvarea numelor	getByName(), getAllByName(), getLocalHost(), getLoopbackAddress(), getHostAddress(), getHostName()
InetSocketAddress	Adresă IP (Internet Protocol) + port	new InetSocketAddress(host, port), getAddress(), getPort()

Clasă	Scop	Metode principale
ServerSocket	Socket TCP (Transmission Control Protocol) de ascultare (socket + bind + listen)	ServerSocket(port[, backlog[, bindAddr]]), accept(), close(), setSoTimeout()
Socket	Socket TCP (Transmission Control Protocol) conectat (pe partea clientului sau acceptat pe server)	Socket(host, port), getInputStream(), getOutputStream(), shutdownOutput(), setSoTimeout(), setTcpNoDelay(), close()
DatagramSocket	Socket UDP (User Datagram Protocol)	DatagramSocket([port]), send(), receive(), connect(), setSoTimeout(), setBroadcast(), close()
DatagramPacket	O datagramă UDP (User Datagram Protocol): buffer de date + lungime + adresă/port la distanță	getData(), getLength(), getAddress(), getPort(), setLength()
MulticastSocket	Socket UDP (User Datagram Protocol) care se poate înscrie în grupuri multicast	joinGroup(), leaveGroup()

Tabelul 5. *Excepții de rețea frecvente în Java*

Excepție	Cauză tipică
UnknownHostException	Numele gazdei nu poate fi rezolvat (eroare DNS (Domain Name System – sistemul numelor de domeniu), greșeală de scriere)
ConnectException: Connection refused	Niciun proces nu ascultă pe portul destinație: gazda serverului a răspuns cu RST (Reset – resetare)
SocketTimeoutException	Un apel blocant (connect, accept, read, receive) a depășit temporizarea setată cu setSoTimeout()
BindException: Address already in use	Un alt socket (sau o rulare anterioară a aceluiași program) folosește deja portul
BindException: Permission denied	În Linux, asocierea cu un port < 1024 fără drepturi de root
SocketException: Connection reset	Celălalt capăt a abandonat conexiunea (RST (Reset – resetare)), de ex. procesul a fost oprit forțat
PortUnreachableException	Un DatagramSocket conectat a primit un mesaj ICMP (Internet Control Message Protocol) Port Unreachable

Listingul 1. Lucrul cu InetAddress

```
import java.net.*;

public class AddressInfo {
    public static void main(String[] args) throws Exception {
        String name = (args.length > 0) ? args[0] : "www.example.com";
        for (InetAddress a : InetAddress.getAllByName(name))     // rezolvare DNS (Domain Name System –
sistemul numelor de domeniu) (înregistrări A și AAAA)
            System.out.println(name + " -> " + a.getHostAddress());
        System.out.println("Gazda locala: " + InetAddress.getLocalHost());
        System.out.println("Loopback    : " + InetAddress.getLoopbackAddress());
    }
}
```

6. Comunicația client-server TCP (Transmission Control Protocol) în Java

6.1 Pașii

Tabelul 6. *Pașii unui server și ai unui client TCP (Transmission Control Protocol)*

Server TCP (Transmission Control Protocol)	Client TCP (Transmission Control Protocol)
1. Se creează un `ServerSocket` pe un port cunoscut (socket + bind + listen).	1. Se creează un `Socket(host, port)` — acesta realizează stabilirea conexiunii în trei pași.
2. Se apelează `accept()`; blochează până se conectează un client și întoarce un `Socket` conectat.	2. Se obțin fluxurile de intrare și de ieșire ale socket-ului.
3. Se obțin fluxurile de intrare/ieșire ale socket-ului conectat.	3. Se trimite o cerere și se citește răspunsul, conform protocolului de aplicație.
4. Se citesc cererile, se procesează, se scriu răspunsurile.	4. Se închide socket-ul (se trimite FIN (Finish – terminare)).
5. Se închide socket-ul conectat; se revine la pasul 2 pentru următorul client.	

6.2 Un server TCP (Transmission Control Protocol) de ecou concurent

Serverul de mai jos ascultă pe un port dat în linia de comandă (implicit 5000). Pentru fiecare client acceptat pornește un fir de execuție nou, astfel încât mai mulți clienți pot fi serviți simultan. *Protocolul de aplicație* este bazat pe linii: clientul trimite o linie de text; serverul răspunde cu `ECHO:` urmat de linia scrisă cu majuscule; linia `BYE` încheie sesiunea.

Listingul 2. TcpEchoServer.java

```
import java.io.*;
import java.net.*;

public class TcpEchoServer {
    public static void main(String[] args) throws IOException {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 5000;
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Server ecou (socket stream) asculta pe portul " + port);
            while (true) {
                Socket client = server.accept();          // blocheaza pana se conecteaza un client
                new Thread(() -> handle(client)).start(); // cate un fir de executie pentru fiecare client
            }
        }
    }

    private static void handle(Socket s) {
        String who = s.getInetAddress().getHostAddress() + ":" + s.getPort();
        System.out.println("Conectat: " + who);
        try (s;
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(s.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true)) {
            String line;
            while ((line = in.readLine()) != null) {
                if (line.equalsIgnoreCase("BYE")) { out.println("Goodbye"); break; }
                out.println("ECHO: " + line.toUpperCase());
            }
        } catch (IOException e) {
            System.out.println("Eroare la " + who + ": " + e.getMessage());
        }
        System.out.println("Deconectat: " + who);
    }
}
```

Listingul 3. TcpEchoClient.java

```
import java.io.*;
import java.net.*;
```

```
public class TcpEchoClient {
    public static void main(String[] args) throws IOException {
        String host = (args.length > 0) ? args[0] : "localhost";
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 5000;
        try (Socket s = new Socket(host, port);   // aici are loc stabilirea conexiunii in trei pasi
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(s.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true);
             BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
            System.out.println("Conectat la " + host + ":" + port
                               + " de pe portul local " + s.getLocalPort());
            String line;
            while ((line = kb.readLine()) != null) {
                out.println(line);
                String reply = in.readLine();
                if (reply == null) break;
                System.out.println(reply);
                if (line.equalsIgnoreCase("BYE")) break;
            }
        }
    }
}
```

6.3 Explicații

- `new ServerSocket(port)` creează socket-ul de ascultare; portul apare ca LISTEN în `netstat` / `ss`. Backlog-ul (implicit 50) este lungimea cozii conexiunilor stabilite care așteaptă `accept()`.
- `accept()` blochează; când revine, stabilirea conexiunii este deja încheiată. Socket-ul întors are același port local ca socket-ul serverului — se deosebește prin adresa și portul clientului.
- `new Socket(host, port)` rezolvă numele, alege un port local efemer și realizează stabilirea conexiunii. Dacă nu ascultă niciun server, aruncă `ConnectException`.
- TCP (Transmission Control Protocol) este un **flux de octeți**. `BufferedReader.readLine()` și `PrintWriter.println()` impun peste el o delimitare *pe linii*: fiecare mesaj se termină cu un caracter de linie nouă.
- `PrintWriter(..., true)` activează **golirea automată (auto-flush)** la `println()`. Fără golire, datele rămân în buffer și celălalt capăt așteaptă la nesfârșit — o eroare clasică.
- `readLine()` întoarce `null` când celălalt capăt închide conexiunea (sfârșitul fluxului, s-a primit FIN (Finish – terminare)).
- **try-with-resources** închide automat fluxurile și socket-urile, chiar și după o excepție. Specificați întotdeauna setul de caractere (UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți)), pentru ca sistemele Windows și Linux să interopereze.

6.4 Servere iterative comparativ cu servere concurente

Tabelul 7. *Modele de concurență pentru servere*

Model	Cum funcționează	Avantaje	Dezavantaje
Iterativ	accept → servește complet clientul → accept pentru următorul	Foarte simplu; fără stare partajată	Un client lent îi blochează pe toți ceilalți
Un fir de execuție pe client	accept → pornește un fir nou pentru client (listingul 2)	Simplu; clienții sunt serviți în paralel	Mii de clienți înseamnă mii de fire de execuție ale platformei
Bazin de fire (thread pool)	accept → trimite clientul unui ExecutorService	Consum limitat de resurse	Dimensiunea bazinului limitează numărul de clienți simultani
Fire virtuale (Java 21+)	accept → câte un fir virtual ieftin pentru fiecare client	Cod simplu, scalează la foarte mulți clienți	Necesită un JDK (Java Development Kit – kit de dezvoltare Java) recent

Model	Cum funcționează	Avantaje	Dezavantaje
Bazat pe evenimente (NIO (New Input/Output – noua bibliotecă de intrare/ieșire))	Un singur fir multiplexează multe canale neblocante cu un Selector	Scalabilitate maximă	Cod mai complex

Listingul 4. Varianta cu bazin de fire a buclei accept

```
ExecutorService pool = Executors.newFixedThreadPool(50);   // sau
Executors.newVirtualThreadPerTaskExecutor() (Java 21+)
try (ServerSocket server = new ServerSocket(5000)) {
    while (true) {
        Socket client = server.accept();
        pool.submit(() -> handle(client));
    }
}
```

6.5 Delimitarea mesajelor peste TCP (Transmission Control Protocol)

Deoarece TCP (Transmission Control Protocol) nu păstrează limitele mesajelor, aplicația trebuie să definească unde se termină un mesaj. Trei tehnici uzuale sunt: un **delimitator** (de ex. caracterul de linie nouă, ca în antetele HTTP (Hypertext Transfer Protocol), SMTP (Simple Mail Transfer Protocol) sau protocolul nostru de ecou), un **prefix de lungime** (protocoale binare) sau mesaje de **dimensiune fixă**. Un schimb cu prefix de lungime în Java:

Listingul 5. Delimitare cu prefix de lungime

```
// emițător
DataOutputStream out = new DataOutputStream(new BufferedOutputStream(socket.getOutputStream()));
byte[] msg = text.getBytes(StandardCharsets.UTF_8);
out.writeInt(msg.length);   // lungime pe 4 octeți, big-endian (ordinea octeților în rețea)
out.write(msg);
out.flush();

// receptor
DataInputStream in = new DataInputStream(new BufferedInputStream(socket.getInputStream()));
int len = in.readInt();
byte[] buf = new byte[len];
in.readFully(buf);          // repetă până sosesc exact len octeți
String text = new String(buf, StandardCharsets.UTF_8);
```

6.6 Opțiuni utile ale socket-urilor

Tabelul 8. *Opțiuni ale socket-urilor TCP (Transmission Control Protocol)*

Opțiune (metodă Java)	Efect
setSoTimeout(ms)	Timpul maxim de blocare pentru read()/accept()/receive(); apoi SocketTimeoutException
connect(addr, timeoutMs)	Limitează timpul petrecut cu stabilirea conexiunii (folosiți `new Socket()` și apoi `connect()`)
setTcpNoDelay(true)	Dezactivează algoritmul lui Nagle: scrierile mici se trimit imediat (protocoale interactive, jocuri)
setKeepAlive(true)	Trimite periodic sonde keep-alive pe conexiunile inactive, pentru a detecta capetele căzute
setReuseAddress(true)	Permite unui server să-și reasocieze portul cât timp conexiunile vechi sunt în TIME_WAIT
setReceiveBufferSize / setSendBufferSize	Modifică dimensiunea bufferelor din nucleu — influențează fereastra de recepție anunțată

Opțiune (metodă Java)	Efect
shutdownOutput()	Semi-închidere: trimite FIN (Finish – terminare), dar continuă să citească răspunsul celuilalt capăt

7. Comunicația client-server UDP (User Datagram Protocol) în Java

7.1 Pașii

Tabelul 9. *Pașii unui server și ai unui client UDP (User Datagram Protocol)*

Server UDP (User Datagram Protocol)	Client UDP (User Datagram Protocol)
1. Se creează un `DatagramSocket(port)` pe un port cunoscut.	1. Se creează un `DatagramSocket()` — OS (Operating System – sistem de operare) alege un port efemer.
2. Se pregătește un `DatagramPacket` gol, cu un buffer, și se apelează `receive()` — blochează până sosește o datagramă.	2. Se construiește un `DatagramPacket` cu datele, adresa și portul serverului; se apelează `send()`.
3. Se citesc datele (`getData()`, `getLength()`) și adresa emițătorului (`getAddress()`, `getPort()`).	3. Se apelează `receive()` pentru răspuns, cu o temporizare (`setSoTimeout`), deoarece cererea sau răspunsul se pot pierde.
4. Se construiește un pachet de răspuns adresat emițătorului și se trimite cu `send()`; se revine la pasul 2.	4. Se închide socket-ul.

Listingul 6. UdpEchoServer.java

```
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UdpEchoServer {
    public static void main(String[] args) throws Exception {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 6000;
        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("Server ecou (socket datagrama) asculta pe portul " + port);
            byte[] buf = new byte[1024];
            while (true) {
                DatagramPacket req = new DatagramPacket(buf, buf.length);
                socket.receive(req);                        // blocheaza pana soseste o datagrama
                String msg = new String(req.getData(), 0, req.getLength(), StandardCharsets.UTF_8);
                System.out.println("De la " + req.getAddress().getHostAddress() + ":"
                                   + req.getPort() + " -> " + msg);
                byte[] ans = ("ECHO: " + msg.toUpperCase()).getBytes(StandardCharsets.UTF_8);
                socket.send(new DatagramPacket(ans, ans.length, req.getAddress(), req.getPort()));
            }
        }
    }
}
```

Listingul 7. UdpEchoClient.java

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UdpEchoClient {
    public static void main(String[] args) throws Exception {
        InetAddress server = InetAddress.getByName((args.length > 0) ? args[0] : "localhost");
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 6000;
        try (DatagramSocket socket = new DatagramSocket();   // port local efemer, fara conexiune
             BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
            socket.setSoTimeout(2000);                       // UDP (User Datagram Protocol) nu ofera
garantii: folosim o temporizare
            String line;
            while ((line = kb.readLine()) != null && !line.equalsIgnoreCase("BYE")) {
                byte[] data = line.getBytes(StandardCharsets.UTF_8);
                socket.send(new DatagramPacket(data, data.length, server, port));
```

```
                byte[] buf = new byte[1024];
                DatagramPacket resp = new DatagramPacket(buf, buf.length);
                try {
                    socket.receive(resp);
                    System.out.println(new String(resp.getData(), 0, resp.getLength(),
                                                  StandardCharsets.UTF_8));
                } catch (SocketTimeoutException e) {
                    System.out.println("Niciun raspuns in 2 s (datagrama sau raspuns pierdut?)");
                }
            }
        }
    }
}
```

7.2 Explicații

- **Nu există conexiune și nici accept()**: un singur socket de server primește datagrame de la toți clienții; răspunsul pleacă spre adresa și portul găsite în pachetul primit.
- Fiecare `send()` produce exact o datagramă, iar fiecare `receive()` întoarce exact una — **limitele mesajelor sunt păstrate**, deci nu este nevoie de delimitare.
- Folosiți întotdeauna `getLength()`: bufferul poate fi mai mare decât datagrama și poate conține încă octeți dintr-un mesaj anterior, mai lung.
- Dacă bufferul este mai mic decât datagrama, octeții în plus sunt **eliminați fără avertisment**.
- Încărcătura utilă UDP (User Datagram Protocol) maximă peste IPv4 (Internet Protocol versiunea 4) este de 65.507 octeți, dar datagramele mai mari de aproximativ 1.472 de octeți (MTU (Maximum Transmission Unit – unitatea maximă de transmisie) Ethernet 1500 − 20 − 8) sunt fragmentate de IP (Internet Protocol); pierderea unui singur fragment duce la pierderea întregii datagrame. Păstrați datagramele mici.
- **Fiabilitatea este responsabilitatea aplicației:** clientul folosește o temporizare; un client robust retransmite și un număr limitat de ori și pune în fiecare mesaj un identificator de cerere, pentru a asocia răspunsurile cu cererile.
- `DatagramSocket.connect(addr, port)` nu trimite nimic; doar fixează partenerul, filtrează datagramele de la alții și permite erorilor ICMP (Internet Control Message Protocol) să apară ca `PortUnreachableException`.

8. Programarea TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol); bune practici

Tabelul 10. *Comparație între programarea TCP (Transmission Control Protocol) și UDP (User Datagram Protocol)*

Aspect	TCP (Transmission Control Protocol) (socket-uri stream)	UDP (User Datagram Protocol) (socket-uri datagramă)
Clase Java	ServerSocket, Socket	DatagramSocket, DatagramPacket
Stabilirea conexiunii	Da: constructorul Socket / accept()	Nu
Socket-uri pe server	1 de ascultare + 1 pentru fiecare client	1 pentru toți clienții
Unitatea de date	Flux de octeți (necesită delimitare)	Datagramă (limitele sunt păstrate)
Fiabilitate, ordonare	Garantate de TCP (Transmission Control Protocol)	Trebuie adăugate de aplicație, dacă e nevoie
Interfața I/O (Input/Output – intrare/ieșire)	InputStream / OutputStream	send() / receive() de pachete
Sfârșitul comunicației	Detectat: read întoarce -1 / null	Nedetectat: se folosesc temporizări sau mesaje explicite
Broadcast / multicast	Imposibil	Posibil

Aspect	TCP (Transmission Control Protocol) (socket-uri stream)	UDP (User Datagram Protocol) (socket-uri datagramă)
Cost suplimentar și latență	Mai mari (stabilirea conexiunii, ACKs (Acknowledgments – confirmări), antete de 20+ octeți)	Mai mici (antet de 8 octeți, fără stabilirea conexiunii)

Tabelul 11. *Erori tipice în programele cu socket-uri*

Simptom	Cauză probabilă	Remediu
Clientul se blochează definitiv după trimitere	Lipsește golirea bufferului sau lipsește caracterul de linie nouă când celălalt capăt folosește readLine()	Folosiți println() cu auto-flush sau apelați flush()
Al doilea client așteaptă până pleacă primul	Server iterativ	Serviți fiecare client într-un fir de execuție / bazin de fire
BindException la pornirea serverului	Port în uz (alt program sau o instanță anterioară)	Opriți celălalt proces, alegeți alt port, folosiți setReuseAddress
Funcționează pe localhost, dar nu între două PCs (Personal Computers – calculatoare personale)	Firewall-ul blochează portul sau serverul este asociat cu 127.0.0.1	Permiteți portul în firewall; asociați cu 0.0.0.0 (implicit)
Caractere fără sens la sfârșitul unui mesaj UDP (User Datagram Protocol)	Folosirea getData() fără getLength()	new String(p.getData(), 0, p.getLength(), UTF_8)
Caractere cu diacritice deformate între Windows și Linux	Setul de caractere implicit diferă	Specificați întotdeauna explicit UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți)
Serverul are multe socket-uri în CLOSE_WAIT	Serverul nu închide niciodată socket-urile acceptate	Folosiți try-with-resources

9. Rezumat și termeni-cheie

- Un socket este un capăt de comunicație gestionat de OS (Operating System – sistem de operare), identificat prin (adresă IP (Internet Protocol), port, protocol); aplicațiile îl folosesc prin API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket.
- Socket-urile se clasifică după familia de adrese, tip (stream, datagramă, raw, seqpacket), rol, modelul I/O (Input/Output – intrare/ieșire) și modul de livrare.
- Server TCP (Transmission Control Protocol): ServerSocket → accept → fluxuri → close; client TCP (Transmission Control Protocol): Socket(host, port) → fluxuri → close.
- Serverul și clientul UDP (User Datagram Protocol) folosesc amândoi DatagramSocket și DatagramPacket; nu există conexiune, iar aplicația tratează pierderile.
- TCP (Transmission Control Protocol) necesită delimitare (delimitatori sau prefixe de lungime); UDP (User Datagram Protocol) păstrează limitele mesajelor.
- Serverele concurente folosesc fire de execuție, bazine de fire, fire virtuale sau selectori NIO (New Input/Output – noua bibliotecă de intrare/ieșire).

Termeni-cheie: socket, adresă de socket, handle / descriptor de fișier, Berkeley sockets, Winsock, familie de adrese, socket stream, socket datagramă, socket raw, socket de ascultare, socket conectat, backlog, I/O (Input/Output – intrare/ieșire) blocant / neblocant, server iterativ / concurent, delimitarea mesajelor, protocol de aplicație, temporizare.

Partea a II-a — Seminar: analiză, depanare și proiectarea protocoalelor

Problema 1 — Ce apel produce ce pachete?

Pentru programele TCP (Transmission Control Protocol) de ecou (listingurile 2 și 3), precizați ce apel Java declanșează fiecare dintre următoarele și în ce fir/proces are loc: (a) segmentul SYN (Synchronize – sincronizare); (b) momentul în care revine `accept()`; (c) primul segment de date; (d) FIN (Finish – terminare)-ul clientului; (e) un RST (Reset – resetare) în locul unui SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare).

> **Rezolvare**
>
> (a) `new Socket(host, port)` în client (apelează connect()).
>
> (b) Nucleul serverului finalizează singur stabilirea conexiunii când sosește ACK (Acknowledgment – confirmare)-ul final; apoi `accept()` scoate noua conexiune din coadă. Stabilirea se poate încheia chiar înainte de apelul `accept()`.
>
> (c) `out.println(line)` în client (auto-flush împinge octeții către TCP (Transmission Control Protocol)).
>
> (d) Închiderea socket-ului clientului la sfârșitul blocului try-with-resources (sau `shutdownOutput()`).
>
> (e) Clientul se conectează la un port pe care nu ascultă niciun server; clientul primește `ConnectException: Connection refused`.

Problema 2 — Numărarea socket-urilor

Serverul TCP (Transmission Control Protocol) de ecou rulează pe `192.168.1.100:5000`. Sunt conectați trei clienți: doi de la `192.168.1.21` (porturile 50510 și 50511) și unul de la `192.168.1.35` (portul 61200). (a) Câte socket-uri are deschise procesul server? (b) Scrieți 5-tuplul fiecărei conexiuni. (c) Ce afișează pe server `netstat -an` (Windows) sau `ss -tan` (Linux) pentru portul 5000?

> **Rezolvare**
>
> (a) **4 socket-uri**: 1 `ServerSocket` de ascultare + 3 obiecte `Socket` conectate. Toate au portul local 5000.
>
> (b) (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.21, 50510); (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.21, 50511); (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.35, 61200).
>
> (c) O linie `0.0.0.0:5000 … LISTEN` și trei linii `192.168.1.100:5000 <client> ESTABLISHED`.

Problema 3 — Găsiți eroarea

Fiecare fragment de mai jos se compilează, dar nu funcționează corect. Găsiți problema și corectați-o.

Fragment (a)

```
// (a) client
PrintWriter out = new PrintWriter(socket.getOutputStream());
out.println("HELLO");
String reply = in.readLine();          // blochează la nesfârșit
```

Fragment (b)

```
// (b) server, clientul folosește readLine()
out.print("ECHO: " + line);           // out are auto-flush activat
```

Fragment (c)

```
// (c) server UDP (User Datagram Protocol)
socket.receive(packet);
String msg = new String(packet.getData(), StandardCharsets.UTF_8);
```

Fragment (d)

```
// (d) server UDP (User Datagram Protocol) care refolosește un singur obiect pachet
byte[] buf = new byte[1024];
DatagramPacket packet = new DatagramPacket(buf, buf.length);
while (true) {
    socket.receive(packet);           // după un mesaj de 5 octeți, mesajele următoare sunt trunchiate la 5
octeți
    handle(packet);
}
```

Fragment (e)

```
// (e) server
while (true) {
    Socket s = server.accept();
    serveUntilBye(s);                 // ceilalți clienți așteaptă
}
```

Rezolvare

(a) Fără auto-flush: „HELLO” rămâne în buffer. Folosiți `new PrintWriter(socket.getOutputStream(), true)` sau apelați `out.flush()`.

(b) `print` nu trimite caracterul de linie nouă, deci `readLine()` din client nu se termină niciodată; auto-flush funcționează doar cu `println`. Folosiți `out.println(...)`.

(c) `getData()` întoarce întregul buffer de 1024 de octeți, inclusiv octeți vechi sau zero. Folosiți `new String(packet.getData(), 0, packet.getLength(), UTF_8)`.

(d) `receive()` setează lungimea pachetului la dimensiunea datagramei primite; următorul `receive()` poate memora cel mult atâția octeți. Apelați `packet.setLength(buf.length)` înaintea fiecărui `receive()` sau creați un pachet nou în buclă.

(e) Server iterativ: un client îi blochează pe ceilalți. Serviți fiecare client într-un fir de execuție separat sau într-un bazin de fire.

Problema 4 — Ce vede programul?

Tabelul 12. *Comportamentul programelor TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) la erori*

Situație	Client / server TCP (Transmission Control Protocol)	Client / server UDP (User Datagram Protocol)
Serverul nu rulează (aceeași gazdă)	ConnectException: Connection refused (RST (Reset – resetare))	send() reușește; se întoarce ICMP (Internet Control Message Protocol) Port Unreachable; un socket neconectat îl ignoră → receive() expiră; un socket conectat primește PortUnreachableException
Gazda serverului este oprită sau pachetele sunt aruncate de un firewall	connect() așteaptă, apoi expiră (retransmisii SYN (Synchronize – sincronizare))	receive() expiră
Serverul pornit de două ori pe același port	BindException: Address already in use	BindException: Address already in use

Situație	Client / server TCP (Transmission Control Protocol)	Client / server UDP (User Datagram Protocol)
Serverul oprit forțat în timpul sesiunii	Client: read întoarce null / -1 (FIN (Finish – terminare)) sau SocketException: Connection reset (RST (Reset – resetare))	Client: receive() expiră

Discuție: de ce un emițător UDP (User Datagram Protocol) *nu* află de pierderea datagramei sale, în timp ce un emițător TCP (Transmission Control Protocol) știe întotdeauna (în cele din urmă) dacă datele au ajuns?

Problema 5 — Proiectarea unui protocol de aplicație

Proiectați un protocol text pentru un **serviciu de calculator** peste TCP (Transmission Control Protocol). Clientul trimite operații cu două numere; serverul întoarce rezultatul sau o eroare. Apoi adaptați proiectul pentru UDP (User Datagram Protocol).

> **O soluție posibilă**
>
> **Cereri** (câte o linie, terminată cu \n): `ADD a b`, `SUB a b`, `MUL a b`, `DIV a b`, `QUIT`.
>
> **Răspunsuri** (o linie): `OK <rezultat>` sau `ERR <mesaj>` — de ex. `ERR division by zero`, `ERR unknown operation POW`, `ERR operands must be numbers`.
>
> **Sesiune:** după conectare, clientul poate trimite oricâte cereri; la `QUIT` se răspunde cu `OK bye`, iar serverul închide conexiunea.
>
> **Varianta UDP (User Datagram Protocol):** fiecare cerere trebuie să fie de sine stătătoare și să conțină un identificator, de ex. `17 ADD 3 4` → `17 OK 7`. Clientul retransmite după expirarea temporizării (de cel mult 3 ori) și ignoră răspunsurile al căror identificator nu se potrivește. Deoarece ADD/SUB/MUL/DIV sunt *idempotente*, execuția repetată cauzată de retransmisie nu face rău.
>
> O implementare de referință a serverului TCP (Transmission Control Protocol) este dată în anexă.

Problema 6 — Delimitarea mesajelor

Un client TCP (Transmission Control Protocol) execută `out.write("HELLO".getBytes());` `out.write("WORLD".getBytes());`. Serverul execută o singură dată `int n = in.read(buf);`. Enumerați valorile posibile ale lui n și conținutul lui buf. De ce rezultatul nu este determinist? Propuneți două metode prin care serverul să primească exact cele două cuvinte.

> **Rezolvare**
>
> n poate avea orice valoare între 1 și 10: „HELLOWORLD” într-o singură citire, sau „HELLO”, sau „HEL” etc. TCP (Transmission Control Protocol) livrează un flux; limitele segmentelor depind de algoritmul lui Nagle, dimensiunile bufferelor, MSS (Maximum Segment Size – dimensiunea maximă a segmentului) și sincronizare, iar citirea receptorului întoarce ce este disponibil.
>
> Soluții: (1) un delimitator — se trimite `HELLO\nWORLD\n` și se citește cu `readLine()`; (2) un prefix de lungime — `writeInt(5)` + octeți, citire cu `readInt()` + `readFully()` (listingul 5).

Exerciții propuse (temă)

1. Rescrieți serverul TCP (Transmission Control Protocol) de ecou astfel încât să fie iterativ. Rulați doi clienți și descrieți ce se întâmplă.
2. Modificați clientul TCP (Transmission Control Protocol) de ecou astfel încât să seteze o temporizare de conectare de 3 s și o temporizare de citire de 5 s. Testați-l cu o adresă inaccesibilă.

3. Scrieți un client UDP (User Datagram Protocol) care trimite cât mai repede 100 de datagrame numerotate și un server care le numără și raportează numerele lipsă. Rulați-le între două calculatoare conectate prin Wi-Fi.
4. Explicați de ce un server HTTP/1.1 (Hypertext Transfer Protocol versiunea 1.1) trebuie să cunoască lungimea corpului fiecărui răspuns (Content-Length sau codificare chunked).
5. Comparați numărul de pachete necesar pentru un schimb cerere-răspuns cu TCP (Transmission Control Protocol) (inclusiv stabilirea și eliberarea conexiunii) și cu UDP (User Datagram Protocol).

Partea a III-a — Laborator

Ambele variante folosesc **aceleași programe Java** (listingurile 2, 3, 6 și 7 de la curs) și aceleași teme; diferă doar comenzile sistemului de operare. Fiecare variantă durează aproximativ 100 de minute, plus temele de programare. Lucrați în perechi când sunt necesare două calculatoare.

Tabelul 13. *Harta sarcinilor de laborator*

Sarcină	Scop	Windows	Linux
0	Instalarea JDK (Java Development Kit – kit de dezvoltare Java) și pregătirea unui director de lucru	winget, PowerShell	apt
1	Compilarea și rularea serverului și clientului TCP (Transmission Control Protocol) de ecou	javac / java	javac / java
2	Observarea socket-urilor serverului și ale clienților	netstat, Get-NetTCPConnection	ss, jcmd / jstack
3	Testarea serverului cu un client generic	ncat, Test-NetConnection	nc
4	Rularea clientului și serverului pe două calculatoare	ipconfig, New-NetFirewallRule	ip addr, ufw / firewalld
5	Compilarea și rularea programelor UDP (User Datagram Protocol) de ecou	Get-NetUDPEndpoint, ncat -u	ss -u, nc -u
6	Capturarea traficului ambelor programe	Wireshark (Npcap loopback)	tcpdump
7	Provocarea și explicarea erorilor tipice	—	porturi privilegiate

Laboratorul 7-W: socket-uri Java în Windows (Windows 10/11)

Sarcina W0 — Pregătirea mediului

PowerShell

```
winget install EclipseAdoptium.Temurin.21.JDK     # sau descărcați de pe adoptium.net
# deschideți o fereastră PowerShell NOUĂ, ca să fie actualizată variabila PATH
java -version
javac -version
mkdir C:\cn\week07; cd C:\cn\week07
```

Salvați cele patru programe (TcpEchoServer.java, TcpEchoClient.java, UdpEchoServer.java, UdpEchoClient.java) în `C:\cn\week07`, cu codificarea UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți). Merge orice editor (VS (Visual Studio) Code, IntelliJ IDEA, Notepad++). Dacă `javac` nu este recunoscut, adăugați `C:\Program Files\Eclipse Adoptium\jdk-21…\bin` în PATH.

Sarcina W1 — Rularea serverului și clientului TCP (Transmission Control Protocol) de ecou

Compilare și rulare

```
# Fereastra PowerShell 1
javac TcpEchoServer.java TcpEchoClient.java
java TcpEchoServer 5000


# Fereastra PowerShell 2
java TcpEchoClient localhost 5000
salut socketuri
ECHO: SALUT SOCKETURI
BYE
```

Lansare dintr-un singur fișier (JDK (Java Development Kit – kit de dezvoltare Java) 11+)

Pentru teste rapide, o singură clasă poate fi rulată fără compilare prealabilă: `java TcpEchoServer.java 5000`.

La prima pornire a serverului, **Windows Defender Firewall** poate întreba dacă permiteți accesul Java la rețea. Pentru teste pe un singur calculator puteți anula; pentru sarcina W4, permiteți accesul **doar pe rețele private**.

Deschideți încă două ferestre și porniți simultan încă doi clienți. Verificați în fereastra serverului că toți clienții sunt serviți în paralel (câte un fir de execuție fiecare).

Sarcina W2 — Observarea socket-urilor

PowerShell (a treia fereastră)

```
netstat -ano | findstr :5000
Get-NetTCPConnection -LocalPort 5000 | Format-Table
LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess
Get-NetTCPConnection -RemotePort 5000 | Format-Table LocalPort,State,OwningProcess
Get-Process -Id <pid>          # procesul java.exe al serverului
```

Întrebări W2

- Câte linii cu portul local 5000 vedeți când sunt conectați trei clienți? Puneți rezultatul în legătură cu problema 2 de la seminar.
- Ce porturi efemere au primit clienții? Verificați că aparțin intervalului dinamic din Windows.
- Opriți un client cu `BYE` și serverul cu Ctrl+C. Care parte rămâne în TIME_WAIT în fiecare caz?

Sarcina W3 — Testarea cu un client generic

PowerShell

```
Test-NetConnection localhost -Port 5000       # doar stabilirea conexiunii
ncat localhost 5000                            # tastați linii; serverul răspunde (ncat din Nmap)
```

Aceasta arată o proprietate importantă a protocoalelor: **orice** program care respectă protocolul (aici: linii de text peste TCP (Transmission Control Protocol)) poate comunica cu serverul, indiferent de limbajul în care este scris.

Sarcina W4 — Client și server pe două calculatoare

PowerShell

```
# Pe calculatorul SERVER (PowerShell ca administrator)
ipconfig                                          # notați adresa IPv4 (Internet Protocol versiunea 4), de
ex. 192.168.10.21
New-NetFirewallRule -DisplayName "CN Lab port 5000 tcp" -Direction Inbound -Protocol TCP -LocalPort 5000
-Action Allow -Profile Private
New-NetFirewallRule -DisplayName "CN Lab port 6000 udp" -Direction Inbound -Protocol UDP -LocalPort 6000
-Action Allow -Profile Private
java TcpEchoServer 5000

# Pe calculatorul CLIENT
Test-NetConnection 192.168.10.21 -Port 5000
java TcpEchoClient 192.168.10.21 5000

# La final, pe calculatorul SERVER (administrator)
Remove-NetFirewallRule -DisplayName "CN Lab port 5000 tcp"
Remove-NetFirewallRule -DisplayName "CN Lab port 6000 udp"
```

Dacă nu funcționează

- Verificați că profilul rețelei este *Private* (`Get-NetConnectionProfile`).
- Verificați că ambele calculatoare sunt în aceeași rețea (ping poate fi blocat de firewall chiar dacă TCP (Transmission Control Protocol) funcționează).
- Fereastra de dialog a firewall-ului poate să fi creat o regulă *Block* pentru java.exe: căutați-o cu `Get-NetFirewallApplicationFilter | Where-Object Program -like '*java*' | Get-NetFirewallRule` sau în *wf.msc*.

Sarcina W5 — Rularea programelor UDP (User Datagram Protocol) de ecou

PowerShell

```
# Fereastra 1
javac UdpEchoServer.java UdpEchoClient.java
java UdpEchoServer 6000

# Fereastra 2
java UdpEchoClient localhost 6000

# Fereastra 3
Get-NetUDPEndpoint -LocalPort 6000
ncat -u localhost 6000                    # client UDP (User Datagram Protocol) generic
```

Opriți serverul (Ctrl+C) și trimiteți încă o linie din client. Observați mesajul *Niciun raspuns in 2 s*. Porniți din nou serverul: clientul continuă să funcționeze fără reconectare — nu a existat niciodată o conexiune.

Sarcina W6 — Capturarea traficului

În Wireshark, selectați **Adapter for loopback traffic capture** (Npcap) și folosiți filtrul de afișare `tcp.port == 5000 || udp.port == 6000`. Trimiteți o linie cu clientul TCP (Transmission Control Protocol) și una cu clientul UDP (User Datagram Protocol), apoi închideți ambii clienți.

Întrebări W6

- Numărați pachetele pentru o sesiune TCP (Transmission Control Protocol) completă (conectare, o linie, BYE, închidere) și pentru o pereche cerere-răspuns UDP (User Datagram Protocol). Explicați diferența.
- Găsiți textul vostru în încărcătura utilă TCP (Transmission Control Protocol) (*Follow → TCP (Transmission Control Protocol) Stream*). Unde sunt limitele mesajelor?
- Ce segment corespunde lui `new Socket(...)`, care lui `println` și care lui `close()`?

Sarcina W7 — Provocarea erorilor tipice

- Rulați `java TcpEchoClient localhost 5999` fără server → **ConnectException: Connection refused**.
- Porniți de două ori `java TcpEchoServer 5000` → a doua instanță eșuează cu **BindException: Address already in use**.
- Rulați clientul cu un nume de gazdă greșit → **UnknownHostException**.
- Opriți serverul cu Ctrl+C cât timp un client este conectat, apoi tastați o linie în client → răspuns `null` sau **Connection reset**.

Laboratorul 7-L: socket-uri Java în Linux (Ubuntu 22.04/24.04 sau Debian 12)

Sarcina L0 — Pregătirea mediului

Terminal

```
sudo apt update
sudo apt install -y openjdk-21-jdk netcat-openbsd tcpdump iproute2   # openjdk-17-jdk pe versiunile mai
vechi
java -version; javac -version
mkdir -p ~/cn/week07 && cd ~/cn/week07
```

Salvați cele patru programe Java în `~/cn/week07`. Folosiți mai multe terminale (sau `tmux`) pentru a rula simultan servere și clienți.

Sarcina L1 — Rularea serverului și clientului TCP (Transmission Control Protocol) de ecou

Terminal

```
# Terminalul 1
javac TcpEchoServer.java TcpEchoClient.java
java TcpEchoServer 5000


# Terminalul 2
java TcpEchoClient localhost 5000


# Rularea serverului în fundal, într-un singur terminal
java TcpEchoServer 5000 &      # 'jobs' îl listează, 'kill %1' îl oprește
```

Sarcina L2 — Observarea socket-urilor și firelor de execuție

Terminal

```
ss -tan '( sport = :5000 or dport = :5000 )'
ss -tlnp 'sport = :5000'              # socket-ul de ascultare și procesul proprietar
jcmd                                  # listează procesele Java și PIDs (Process Identifiers –
identificatorii proceselor)
jstack <pid> | grep '"Thread-'        # câte un fir pentru fiecare client conectat
ps -o nlwp= -p <pid>                  # numărul total de fire ale JVM (Java Virtual Machine – mașina
virtuală Java)
```

Întrebări L2

- Răspundeți la **Întrebări W2** pentru Linux.
- Conectați trei clienți și verificați cu `jstack` că există trei fire care servesc clienții. Ce se întâmplă cu numărul lor când clienții pleacă?

Sarcina L3 — Testarea cu un client generic

Terminal

```
nc localhost 5000                                  # interactiv
printf 'prima linie\na doua linie\nBYE\n' | nc -N localhost 5000   # automat (-N: închide după EOF (End Of
File – sfârșit de fișier))
```

Sarcina L4 — Client și server pe două calculatoare

Terminal

```
# Pe calculatorul SERVER
ip -4 addr show                          # notați adresa, de ex. 192.168.10.31
sudo ufw allow 5000/tcp; sudo ufw allow 6000/udp          # Ubuntu cu ufw activ
# sau: sudo firewall-cmd --add-port=5000/tcp --add-port=6000/udp   (firewalld, nepermanent)
java TcpEchoServer 5000

# Pe calculatorul CLIENT (Linux sau Windows!)
nc -zv 192.168.10.31 5000
java TcpEchoClient 192.168.10.31 5000

# La final, pe calculatorul SERVER
sudo ufw delete allow 5000/tcp; sudo ufw delete allow 6000/udp
```

> **Test între platforme diferite**
>
> Rulați serverul în Linux și clientul în Windows (sau invers). Deoarece programele specifică explicit UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți) și folosesc un protocol text, ele interoperează fără nicio modificare — rețelei nu îi pasă de sistemul de operare.

Sarcina L5 — Rularea programelor UDP (User Datagram Protocol) de ecou

Terminal

```
# Terminalul 1
javac UdpEchoServer.java UdpEchoClient.java
java UdpEchoServer 6000

# Terminalul 2
java UdpEchoClient localhost 6000

# Terminalul 3
ss -uanp 'sport = :6000'
echo "salut de la nc" | nc -u -w1 localhost 6000
```

Ca în Windows, opriți și reporniți serverul cât timp clientul rulează și observați expirările temporizării.

Sarcina L6 — Capturarea traficului

Terminal

```
sudo tcpdump -i lo -nn -A 'tcp port 5000 or udp port 6000'     # -A afișează încărcătura utilă ca text
```

> **Întrebări L6**
>
> - Răspundeți la **Întrebări W6** folosind rezultatul tcpdump (indicatorii [S], [S.], [.], [P.], [F.]).
> - Unde vedeți textul mesajelor? Produce fiecare `println()` exact un segment?

Sarcina L7 — Erori tipice, inclusiv porturile privilegiate

- Reproduceți cele patru erori din sarcina W7.
- Rulați `java TcpEchoServer 80` ca utilizator obișnuit → **BindException: Permission denied**: în Linux, porturile sub `net.ipv4.ip_unprivileged_port_start` (implicit 1024) sunt rezervate pentru root. Windows nu are o astfel de restricție. Nu rulați serverul ca root; folosiți un port ≥ 1024.
- Porniți serverul, conectați un client, opriți serverul și reporniți-l imediat. Dacă primiți *Address already in use*, verificați `ss -tan state time-wait` și explicați rolul lui `setReuseAddress(true)`.

Teme de programare (ambele variante)

Rezolvați cel puțin **A1 și A2**; A3 și A4 aduc puncte suplimentare. Testați fiecare program pe propriul calculator și între două calculatoare și includeți în referat comenzile de test și rezultatele.

Tabelul 14. *Teme de programare*

#	Temă	Cerințe
A1	Server și client calculator TCP (Transmission Control Protocol)	Implementați protocolul din problema 5 de la seminar (ADD, SUB, MUL, DIV, QUIT; răspunsuri OK / ERR). Serverul trebuie să fie concurent (bazin de fire) și să reziste la date de intrare invalide.
A2	Serviciu de oră UDP (User Datagram Protocol) cu retransmisie	Serverul răspunde la cererea `TIME` cu data și ora curente (`java.time.LocalDateTime.now()`). Clientul așteaptă un răspuns 1 s și reîncearcă de cel mult 3 ori înainte de a raporta eșecul. Testați-l oprind serverul.
A3	Transfer de fișiere TCP (Transmission Control Protocol)	Clientul trimite numele și dimensiunea unui fișier, apoi conținutul acestuia, folosind delimitarea cu prefix de lungime (listingul 5). Serverul îl salvează într-un director `received/` și răspunde cu `OK <octeți>`. Comparați fișierul primit cu originalul (fc /b în Windows, cmp în Linux).
A4	Chat cu mai mulți clienți (bonus)	Fiecare linie trimisă de un client este difuzată tuturor clienților conectați, precedată de numele emițătorului. Păstrați scriitorii într-o colecție sigură la acces concurent (de ex. CopyOnWriteArrayList) și eliminați clienții care se deconectează.

Criterii de evaluare

- Comportament corect conform protocolului, inclusiv în cazurile de eroare (40 %).
- Robustețe: temporizări, închiderea resurselor, fără căderi la date invalide sau deconectări bruște (25 %).
- Teste pe un calculator și pe două calculatoare, cu comenzile OS (Operating System – sistem de operare) folosite pentru verificare (20 %).
- Claritatea codului și comentarii (15 %).

Model de referat de laborator

Tabelul 15. *Structura referatului de laborator*

Secțiune	Conținut
1. Identificare	Nume, grupă, dată, variantă (Windows / Linux), versiunile OS (Operating System – sistem de operare) și JDK (Java Development Kit – kit de dezvoltare Java)
2. Ecou TCP (Transmission Control Protocol)	Comenzile și rezultatele pentru sarcinile 1–3; lista socket-urilor cu trei clienți; răspunsurile la Întrebări 2
3. Două calculatoare	Adresele IP (Internet Protocol), comenzile de firewall, dovada unei sesiuni reușite la distanță
4. Ecou UDP (User Datagram Protocol)	Rezultatele sarcinii 5; comportamentul când serverul este oprit
5. Analiza traficului	Captura (imagine sau text) și numărul de pachete pentru TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol); răspunsurile la Întrebări 6
6. Erori	Fiecare eroare provocată, împreună cu cauza ei
7. Teme	Codul sursă, comenzile de test și rezultatele pentru A1, A2 (și A3, A4)

Anexă — Soluția de referință pentru A1 (pentru cadrul didactic)

Un server calculator concurent care folosește un bazin de fire. Poate fi testat cu clientul TCP (Transmission Control Protocol) de ecou din listingul 3 (`java TcpEchoClient localhost 5001`) sau cu `ncat` / `nc`.

Listingul 8. CalcServer.java

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

/** Protocol: cerere "OP a b\n" cu OP din {ADD,SUB,MUL,DIV};
 *  răspuns "OK rezultat\n" sau "ERR mesaj\n". */
public class CalcServer {
    public static void main(String[] args) throws IOException {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 5001;
        ExecutorService pool = Executors.newFixedThreadPool(20);
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Server calculator pe portul " + port);
            while (true) {
                Socket c = server.accept();
                pool.submit(() -> serve(c));
            }
        }
    }

    static void serve(Socket c) {
        try (c;
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(c.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(c.getOutputStream(), "UTF-8"), true)) {
            String line;
            while ((line = in.readLine()) != null) {
                if (line.trim().equalsIgnoreCase("QUIT")) { out.println("OK bye"); break; }
                out.println(compute(line.trim()));
            }
        } catch (IOException e) { /* clientul a disparut */ }
    }

    static String compute(String req) {
        String[] p = req.split("\\s+");
        if (p.length != 3) return "ERR usage: OP a b";
        try {
            double a = Double.parseDouble(p[1]), b = Double.parseDouble(p[2]);
            switch (p[0].toUpperCase()) {
                case "ADD": return "OK " + (a + b);
                case "SUB": return "OK " + (a - b);
                case "MUL": return "OK " + (a * b);
                case "DIV": return (b == 0) ? "ERR division by zero" : "OK " + (a / b);
                default:    return "ERR unknown operation " + p[0];
            }
        } catch (NumberFormatException e) {
            return "ERR operands must be numbers";
        }
    }
}
```

Sesiune exemplu (verificată)

```
ADD 3 4        ->  OK 7.0
DIV 1 0        ->  ERR division by zero
POW 2 3        ->  ERR unknown operation POW
MUL 2.5 4      ->  OK 10.0
QUIT           ->  OK bye
```

Bibliografie

- J. F. Kurose, K. W. Ross — *Computer Networking: A Top-Down Approach*, ed. a 8-a, Pearson, 2021 — secțiunile 2.7 (programarea cu socket-uri) și 3.
- E. R. Harold — *Java Network Programming*, ed. a 4-a, O'Reilly, 2013.
- W. R. Stevens, B. Fenner, A. M. Rudoff — *Unix Network Programming, Vol. 1: The Sockets Networking API (Application Programming Interface – interfață de programare a aplicațiilor)*, ed. a 3-a, Addison-Wesley, 2003.
- Oracle — documentația API (Application Programming Interface – interfață de programare a aplicațiilor) Java SE (Standard Edition) pentru pachetele java.net și java.nio.channels; The Java Tutorials: *All About Sockets* și *All About Datagrams*.
- Microsoft Learn — Windows Sockets 2 (Winsock); New-NetFirewallRule; Get-NetTCPConnection. Pagini de manual Linux: socket(7), tcp(7), udp(7), ss(8), nc(1).